Chapter 4: Math, Characters, and Strings

COMP 110
Worked examples, discussion, and quizzes

Learning goals

Use Math methods and explain rounding behavior
Convert representations among common bases
Distinguish char values from Unicode characters
Format readable output with printf
Read, compare, search, and slice strings
Validate indexes before extracting text

Math methods are called through the class

```
double radius = 2;
double area = Math.PI * Math.pow(radius, 2);
System.out.printf(Locale.US, "Area: %.5f%n", area);
System.out.printf(Locale.US, "E: %.5f%n", Math.E);
```

Math.PI and Math.E are constants. Math.pow is a static method.

Java: MathBasics.java

Powers and square roots

```
System.out.println(Math.pow(2, 3));
System.out.println(Math.pow(3, 2));
System.out.printf(Locale.US, "%.5f%n", Math.pow(3.5, 2.5));
System.out.println(Math.sqrt(4));
System.out.printf(Locale.US, "%.2f%n", Math.sqrt(10.5));
System.out.println(Math.sqrt(-1));
```

The formatted values are rounded for display, rather than exact results.

Java: PowersAndRoots.java

Trigonometric methods use radians

```
double radians = Math.toRadians(30);
System.out.printf(Locale.US, "sin: %.3f%n", Math.sin(radians));
System.out.printf(Locale.US, "cos: %.3f%n", Math.cos(0));
System.out.printf(Locale.US, "degrees: %.1f%n",
    Math.toDegrees(Math.PI));
```

Convert an angle in degrees before passing it to sin or cos.

Java: RadiansDemo.java

Ceiling, floor, and a cast differ

```
System.out.println(Math.ceil(2.3));
System.out.println(Math.floor(2.3));
System.out.println(Math.ceil(-2.3));
System.out.println(Math.floor(-2.3));
System.out.println((int) -2.3);
```

Ceiling moves toward positive infinity; floor toward negative infinity.

Java: CeilFloorDemo.java

rint and round use different tie rules

```
System.out.println(Math.rint(2.5));
System.out.println(Math.rint(3.5));
System.out.println(Math.round(2.5));
System.out.println(Math.round(-2.5));
long whole = Math.round(3.2);
System.out.println(whole);
```

rint ties go to an even integer. round ties go toward positive infinity.

Java: RoundingTies.java

Rounding a few values

Input	ceil / floor / rint / round
2.3	3.0 / 2.0 / 2.0 / 2
-2.3	-2.0 / -3.0 / -2.0 / -2
2.5	3.0 / 2.0 / 2.0 / 3
-2.5	-2.0 / -3.0 / -2.0 / -2

Trace the negative values as carefully as the positive ones.

Minimum, maximum, and absolute value

```
System.out.println(Math.max(2, 3));
System.out.println(Math.max(2.5, 3));
System.out.println(Math.min(2.5, 3.6));
System.out.println(Math.abs(-2.1));
System.out.println(Math.abs(Integer.MIN_VALUE));
System.out.println(Math.abs((long) Integer.MIN_VALUE));
```

For the minimum int value, widen before taking its absolute value.

Java: MinMaxAbs.java

Generate a small random integer range

```
double sample = Math.random();
int digit = (int) (sample * 10);
int die = 1 + (int) (Math.random() * 6);
int value = 50 + (int) (Math.random() * 50);
System.out.println("Sample: " + sample);
System.out.println("Digit: " + digit);
System.out.println("Die: " + die);
System.out.println("50-99: " + value);
```

Scale first, cast second, then add the lower bound.

Java: RandomRanges.java

Quiz 1: Math methods

1. Math.floor(-1.2)
2. Math.ceil(-1.2)
3. Math.rint(2.5)
4. Math.round(-2.5)
5. Math.sqrt(9)
6. What is the integer range of a die expression?

Java: MathQuiz.java

A base determines the allowed digits

Base	Digits and place-value multiplier
Binary: 2	0-1; powers of 2
Octal: 8	0-7; powers of 8
Decimal: 10	0-9; powers of 10
Hexadecimal: 16	0-9 and A-F; powers of 16

A-F represent decimal values 10-15. Digits keep the same values in a base.

Place value works in every base

7423 in base 10:
$7 \times 1000 + 4 \times 100 + 2 \times 10 + 3$

1011 in base 2:
$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 = 11$

The rightmost digit has place value 1.

The same values: 0 through 7

Decimal	Binary	Octal	Hex
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7

The value stays the same; its written representation changes.

The same values: 8 through 15

Decimal	Binary	Octal	Hex
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

The value stays the same; its written representation changes.

Binary to decimal: expand the places

```
int value = 1 * 128 + 0 * 64 + 1 * 32 + 0 * 16
    + 1 * 8 + 0 * 4 + 1 * 2 + 1;
System.out.println(value);
System.out.println(Integer.parseInt("10101011", 2));
```

10101011 in base 2 represents decimal 171.

Java: BinaryPlaceValue.java

Octal to decimal: use powers of eight

```
int value = 2 * 512 + 0 * 64 + 7 * 8 + 0;
System.out.println(value);
System.out.println(Integer.parseInt("2070", 8));
System.out.println(Integer.parseInt("10", 8));
```

2070 in base 8 represents 1080 in base 10.

Java: OctalPlaceValue.java

Hexadecimal to decimal

```
System.out.println(7 * 16 + 15);
System.out.println(Integer.parseInt("7F", 16));
System.out.println(Integer.parseInt("FFFF", 16));
```

7F is 7 × 16 + 15. FFFF represents 65535.

Java: HexPlaceValue.java

Decimal 123 to binary: divide by 2

Division	Quotient and remainder
123 / 2, then 61 / 2	61 r1; then 30 r1
30 / 2, then 15 / 2	15 r0; then 7 r1
7 / 2, then 3 / 2	3 r1; then 1 r1
1 / 2	0 r1; stop

Read the remainders in reverse order: 1111011.

Decimal 123 to octal: divide by 8

Division	Quotient and remainder
123 / 8	15 r3
15 / 8	1 r7
1 / 8	0 r1; stop

Read upward: 173 in base 8. Check: 64 + 56 + 3 = 123.

Decimal 123 to hexadecimal: divide by 16

Division	Quotient and remainder
123 / 16	7 r11; write B for digit value 11
7 / 16	0 r7; stop
Read in reverse order	7B

Check: 7 × 16 + 11 = 123.

Binary and hexadecimal group naturally

Each hexadecimal digit represents four bits.

7B in hex = 0111 1011 in binary

10101011 in binary = 1010 1011 = AB in hex

Group binary digits from the right.

Java literals can use different bases

```
int decimal = 123;
int binary = 0b1111011;
int octal = 0173;
int hex = 0x7B;
System.out.println(decimal);
System.out.println(binary);
System.out.println(octal);
System.out.println(hex);
System.out.println(010); // octal literal: decimal 8
```

A leading zero makes an integer literal octal. Avoid accidental 010.

Java: BaseLiterals.java

Parse a numeral with an explicit radix

```
System.out.println(Integer.parseInt("1111011", 2));
System.out.println(Integer.parseInt("173", 8));
System.out.println(Integer.parseInt("7B", 16));
System.out.println(Integer.parseInt("10", 2));
System.out.println(Integer.parseInt("10", 16));
```

The base tells the parser how to interpret the digits.

Java: ParseRadix.java

Format a value in another base

```
int value = 123;
System.out.println(Integer.toBinaryString(value));
System.out.println(Integer.toOctalString(value));
System.out.println(Integer.toHexString(value));
System.out.println(Integer.toString(value, 16));
```

These methods return strings. The int value itself does not change.

Java: FormatRadix.java

Quiz 2: number systems

1. What is 10 in binary, written in decimal?
2. What is 10 in octal, written in decimal?
3. What is 10 in hex, written in decimal?
4. Write decimal 255 in hexadecimal.
5. What does System.out.println(010) print?
6. Which division base converts decimal to octal?

Java: BasesQuiz.java

Discussion: one value, several representations

An int holds the value 123.
Its decimal output is 123.
Its binary text is 1111011.
Its hexadecimal text is 7b.

Did converting the display change the stored number?

char literals use single quotes

```
char letter = 'A';
char digit = '4';
char escaped = '\u0041';
System.out.println(letter);
System.out.println(digit);
System.out.println(escaped);
char next = 'a';
System.out.println(++next);
```

'4' is a char value; 4 is an int value.

Java: CharacterBasics.java

Unicode is larger than one char

A Java char stores one 16-bit UTF-16 code unit.
A String stores a sequence of those code units.

Some Unicode code points need two char values.
A visible symbol can also combine several code points.

String length is not always a visible-symbol count.

One supplementary code point uses two chars

```
String symbol = "\uD83D\uDE00";
System.out.println(symbol.length());
System.out.println(symbol.codePointCount(0, symbol.length()));
System.out.println(Character.isSurrogate(symbol.charAt(0)));
```

The escaped smiley has length 2 but contains one code point.

Java: UnicodeLength.java

ASCII ranges used in these exercises

Characters	Decimal code values
'0' through '9'	48 through 57
'A' through 'Z'	65 through 90
'a' through 'z'	97 through 122
Space / newline / tab	32 / 10 / 9

ASCII values 0-127 occupy the same code points in Unicode.

Escape sequences represent special characters

```
System.out.println("Line 1\nLine 2");
System.out.println("A\tB");
System.out.println("She said \"Hi\".");
System.out.println("C:\\temp\\notes.txt");
System.out.println("Single quote: " + '\'');
```

Use \n for newline, \t for tab, \\ for backslash, and \" for a quote.

Java: EscapeSequences.java

A digit code differs from its numeric value

```
int code = 'a';
char letter = 97;
char digit = '7';
int value = digit - '0';
System.out.println(code);
System.out.println(letter);
System.out.println((int) digit);
System.out.println(value);
```

'7' has code value 55. Subtracting '0' gives digit value 7.

Java: CharNumericConversions.java

char arithmetic is promoted to int

```
char letter = 'a';
System.out.println(letter + 1);
System.out.println((char) (letter + 1));
System.out.println('a' + 'b');
System.out.println("" + 'a' + 'b');
```

Use an explicit conversion or String context when that is the intent.

Java: CharacterArithmetic.java

Range checks describe an ASCII policy

```
if (ch >= 'A' && ch <= 'Z') {
    System.out.println("ASCII uppercase");
} else if (ch >= 'a' && ch <= 'z') {
    System.out.println("ASCII lowercase");
} else if (ch >= '0' && ch <= '9') {
    System.out.println("ASCII digit");
} else {
    System.out.println("Other");
}
```

Check the input length before calling charAt(0).

Java: AsciiCharacterTest.java

Character methods express common tests

```
System.out.println(Character.isDigit('7'));
System.out.println(Character.isLetter('A'));
System.out.println(Character.isLetterOrDigit('!'));
System.out.println(Character.isLowerCase('a'));
System.out.println(Character.isUpperCase('A'));
System.out.println(Character.toLowerCase('A'));
System.out.println(Character.toUpperCase('a'));
```

The method is isLetterOrDigit, with Or in the name.

Java: CharacterMethods.java

Unicode recognition can be broader than ASCII

```
char digit = '\u0663';
System.out.println(Character.isDigit(digit));
System.out.println(digit >= '0' && digit <= '9');
System.out.println(Character.digit(digit, 10));
System.out.println(Character.digit('F', 16));
System.out.println(Character.digit('G', 16));
```

A Unicode decimal digit can pass isDigit and fail the ASCII range test.

Java: UnicodeCharacterTest.java

Quiz 3: chars and code values

1. What is the code value of ASCII zero?
2. What is the result of '8' - '0'?
3. What does 'A' + 1 produce?
4. What does (char) ('A' + 1) produce?
5. Is punctuation a letter or digit?
6. Must a visible symbol fit in one char?

Java: CharactersQuiz.java

printf combines a format and values

```
double amount = 12.345;
System.out.printf(Locale.US, "Amount: $%.2f%n", amount);
System.out.printf("Passed: %b%n", true);
System.out.printf("Grade: %c%n", 'A');
System.out.printf("Count: %d%n", 12);
System.out.printf("Name: %s%n", "Ada");
System.out.printf("Progress: 50%%%n");
```

Each conversion specifier must match the corresponding argument.

Java: PrintfBasics.java

Common format conversions

Conversion	Use in these examples
%d / %c	Decimal integer / character
%f / %e	Fixed-point / scientific floating-point display
%s / %b	String / boolean display
%n / %%	Line separator / literal percent sign

Use %.2f for two fractional digits in fixed-point output.

Width, flags, and precision have distinct roles

```
System.out.printf(Locale.US, "|%8.2f|%n", 12.3);
System.out.printf(Locale.US, "|%-8.2f|%n", 12.3);
System.out.printf(Locale.US, "%+d % d%n", 12, 12);
System.out.printf(Locale.US, "%05d%n", 42);
System.out.printf(Locale.US, "%,.2f%n", 12345.6);
System.out.printf(Locale.US, "|%3d|%n", 12345);
System.out.printf(Locale.US, "%.3s%n", "Java");
```

Width is a minimum. A larger value is not cut down to that width.

Java: FormatFlags.java

A readable report uses consistent columns

```
System.out.printf(Locale.US, "%-10s%5s%9s%n",
    "Item", "Qty", "Price");
System.out.printf(Locale.US, "%-10s%5d%9.2f%n",
    "Notebook", 3, 4.5);
System.out.printf(Locale.US, "%-10s%5d%9.2f%n",
    "Pen", 12, 1.25);
```

Text aligns left; quantities and prices align right.

Java: FormatDemo.java

Discussion: repair a formatting mismatch

The program has a double value of 45.46.
A format uses %d to display it.

Should the repair be an integer cast or %f?
What does the requested output need to mean?

Quiz 4: formatting

1. Where are spaces added by %5d for 42?
2. How does %-5d differ?
3. What does %.2f display for 3.5?
4. Does a field width of 3 truncate 12345?
5. What does %.3s display for "Java"?
6. Does formatting change the stored number?

Java: FormattingQuiz.java

A String represents a sequence of text

```
String message = "Welcome to Java";
String empty = "";
System.out.println(message);
System.out.println(message.length());
System.out.println(empty.length());
System.out.println("A".length());
```

String is a reference type. The empty string has length zero.

Java: StringBasics.java

Class calls and instance calls look different

```
double root = Math.sqrt(9);
String text = "Java";
int size = text.length();
char first = text.charAt(0);
System.out.println(root);
System.out.println(size);
System.out.println(first);
```

Math.sqrt uses a class name; text.length uses a String reference.

Java: StaticAndInstance.java

Indexes for the string "Java"

Index	char at that index
0	J
1	a
2	v
3	a

length() is 4. The last valid char index is length() - 1.

Check bounds before calling charAt

```
String text = "Java";
System.out.println(text.charAt(0));
System.out.println(text.charAt(text.length() - 1));
// text.charAt(4); // invalid: index equals length
// "".charAt(0);   // invalid: empty string
```

An index equal to length is past the last char.

Java: CharAtDemo.java

Concatenation produces a String result

```
String first = "Java";
String joined = first.concat(" is fun");
System.out.println(joined);
System.out.println("Sum: " + 2 + 3);
System.out.println("Sum: " + (2 + 3));
System.out.println(first);
```

Parentheses make numeric addition happen before text concatenation.

Java: StringConcatenation.java

Case conversion and trimming return results

```
String original = "  Java  ";
String trimmed = original.trim();
System.out.println("[" + trimmed + "]");
System.out.println(trimmed.toUpperCase(Locale.ROOT));
System.out.println(trimmed.toLowerCase(Locale.ROOT));
System.out.println("[" + original + "]");
```

trim removes edge code units at or below U+0020, including spaces and tabs.

Java: StringCaseAndTrim.java

Calling a transformation does not reassign a variable

```
String text = "Java";
text.toUpperCase(Locale.ROOT);
System.out.println(text);
text = text.toUpperCase(Locale.ROOT);
System.out.println(text);
```

Keep the returned value when later code needs the transformed text.

Java: StringImmutability.java

Quiz 5: String basics

Assume String text = "Java".

1. text.length()
2. text.charAt(0)
3. text.charAt(3)
4. Is text.charAt(4) valid?
5. Does text.toUpperCase() reassign text?

Java: StringsQuiz.java

next reads one whitespace-delimited token

```
Scanner input = new Scanner(System.in);
System.out.print("Enter three words: ");
String first = input.next();
String second = input.next();
String third = input.next();
System.out.println(first);
System.out.println(second);
System.out.println(third);
```

Spaces and line breaks separate tokens in this Scanner example.

Java: ReadWords.java

nextLine reads the rest of a line

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a full line: ");
String text = input.nextLine();
System.out.println("[" + text + "]");
System.out.println("Length: " + text.length());
```

The returned String excludes the line separator.

Java: ReadLine.java

Mixing token input and line input needs care

```
Scanner input = new Scanner(System.in);
System.out.print("Enter age on its own line: ");
int age = input.nextInt();
input.nextLine(); // consume the rest of that line
System.out.print("Enter full name: ");
String name = input.nextLine();
System.out.println(age + ": " + name);
```

The extra nextLine consumes the remainder of the age line.

Java: ScannerLineTransition.java

Read a first char with an empty-input guard

```
if (text.isEmpty()) {
    System.out.println("Empty input");
} else {
    char first = text.charAt(0);
    System.out.println("First char: " + first);
}
```

Check isEmpty before accessing index zero.

Java: ReadCharacter.java

Discussion: choose an input contract

You need a city name such as New York.
Would you use next or nextLine?

If blank input is allowed, when is charAt(0) safe?
If edge spaces matter, should trim be automatic?

Use equals to compare String contents

```
String first = "Java";
String second = new String("Java");
System.out.println(first == second);
System.out.println(first.equals(second));
System.out.println(first.equalsIgnoreCase("java"));
System.out.println("yes".equals("YES"));
```

== compares references. equals compares the text contents here.

Java: StringEquality.java

compareTo reports a negative, zero, or positive result

```
System.out.println("Ada".compareTo("Bob") < 0);
System.out.println("Java".compareTo("Java"));
System.out.println("Bob".compareTo("Ada") > 0);
System.out.println("JAVA".compareToIgnoreCase("java"));
```

Check the sign. Do not assume the result is always -1 or +1.

Java: StringOrdering.java

Order two city strings

```
if (first.compareTo(second) <= 0) {
    System.out.println(first);
    System.out.println(second);
} else {
    System.out.println(second);
    System.out.println(first);
}
```

The full program reads each city with nextLine.

Java: OrderTwoCities.java

Test prefixes, suffixes, and contained text

```
String filename = "Chapter04.java";
System.out.println(filename.startsWith("Chapter"));
System.out.println(filename.endsWith(".java"));
System.out.println(filename.endsWith(".JAVA"));
System.out.println(filename.contains("04"));
```

These calls are case-sensitive. Choose a case policy deliberately.

Java: PrefixesAndSuffixes.java

substring includes the start and excludes the end

```
String text = "Welcome to Java";
System.out.println(text.substring(0, 7));
System.out.println(text.substring(11));
System.out.println(text.substring(7, 7).length());
System.out.println(text.substring(text.length()).length());
```

For substring(begin, end), the resulting length is end - begin.

Java: SubstringDemo.java

indexOf starts searching at the supplied index

```
String text = "banana";
System.out.println(text.indexOf('a'));
System.out.println(text.indexOf('a', 1));
System.out.println(text.indexOf('a', 2));
System.out.println(text.indexOf("ana"));
System.out.println(text.indexOf('x'));
```

fromIndex is inclusive, rather than strictly after that position.

Java: IndexOfDemo.java

lastIndexOf searches backward

```
String text = "banana";
System.out.println(text.lastIndexOf('a'));
System.out.println(text.lastIndexOf('a', 3));
System.out.println(text.lastIndexOf("ana"));
System.out.println(text.lastIndexOf("ana", 1));
```

The starting position can itself be a match.

Java: LastIndexOfDemo.java

Search, check, then extract

```
int space = text.indexOf(' ');
if (space <= 0 || space == text.length() - 1) {
    System.out.println("Need text on both sides of a space");
} else {
    String first = text.substring(0, space);
    String rest = text.substring(space + 1);
    System.out.println("First: " + first);
    System.out.println("Rest: " + rest);
}
```

A missing separator must be handled before substring is called.

Java: SplitName.java

Parsing and formatting go in opposite directions

```
int count = Integer.parseInt("123");
double amount = Double.parseDouble("12.5");
String countText = String.valueOf(count);
String joined = amount + "";
System.out.println(count + 1);
System.out.println(amount * 2);
System.out.println(countText + 1);
System.out.println(joined);
```

A numeric-looking String still concatenates as text.

Java: StringNumberConversion.java

String.format returns a value to keep or reuse

```
double amount = 12.5;
String report = String.format(Locale.US,
    "Amount: $%.2f", amount);
System.out.println(report);
System.out.println(String.valueOf(42));
```

printf writes output; String.format returns formatted text.

Java: FormattedString.java

Quiz 6: search, slicing, and comparison

Assume String text = "banana".

1. text.indexOf('a', 1)
2. text.lastIndexOf('a', 3)
3. text.substring(1, 3)
4. Does "Java".equals("java") return true?
5. What is Integer.parseInt("12") + 3?

Java: TextOperationsQuiz.java

Case study: one hexadecimal digit

Accept exactly one ASCII digit from:
0-9, A-F, or a-f.

Convert it to an integer from 0 to 15.
Reject empty, multi-character, and invalid input.

Predict the result for F, a, 9, G, and FF.

Hex digit conversion: three valid ranges

```
int value;
if (ch >= '0' && ch <= '9') {
    value = ch - '0';
} else if (ch >= 'A' && ch <= 'F') {
    value = ch - 'A' + 10;
} else if (ch >= 'a' && ch <= 'f') {
    value = ch - 'a' + 10;
} else {
    System.out.println("Invalid hex digit");
    return;
}
System.out.println("Value: " + value);
```

Subtract the start of the range, then add that range's first digit value.

Java: HexDigit2Dec.java

Hex-digit boundary tests

Input	Expected result
0 / 9	0 / 9
A / F	10 / 15
a / f	10 / 15
G, empty line, or FF	Rejected

Check each range endpoint and the input-length guard.

Java: HexDigit2Dec.java

Exit exercise: extract a file extension

Read a simple filename, without a folder path.
Find the last dot.
Require text before the dot and an extension after it.
Display the extension in lowercase.

Try report.final.PDF, README, .profile, and file.

Exit exercise: one possible solution

```
int dot = name.lastIndexOf('.');
if (dot <= 0 || dot == name.length() - 1) {
    System.out.println("No extension");
} else {
    String extension = name.substring(dot + 1);
    System.out.println(extension.toLowerCase(Locale.ROOT));
}
```

Use lastIndexOf, validate its result, then slice after the dot.

Java: FileExtension.java

Chapter review

Check units and rounding rules for Math calls.
Separate a value from its written base representation.
Use an explicit ASCII or Unicode input policy.
Choose format conversions to match the data.
Compare String contents with equals.
Check input length and search results before slicing.